



Kural Tabanlı vs. Öğrenme Tabanlı **Anomali Tespiti**

Over-clustering bulguları ve yüzelsel tespit deneylerimizin karşılaştırmalı analizi.



Geçmişe Dönüş: Hafta 6

Log parse etme sürecindeki tehlikeli bir tuzak: Over-Clustering (Aşırı Kümelenme)

Over-Clustering Sorunu

Farklı Hataların Aynı Şablonda Erimesi

Hafta 6'da Drain3'ün parametrelerini incelerken çok kritik bir durumla karşılaştık. Benzerlik eşiğini (similarity threshold) yanlış ayarladığımızda sistem *over-clustering* yaptı.

Yani; sıradan bir bağlantı kopması logu ile kritik bir veritabanı çökmesi logu, yapısal olarak birbirine benzediği için parser tarafından **aynı template (şablon) kimliği altına** sıkıştırıldı.

Bu durum, anomali tespiti aşamasına geçmeden önce verinin "körleşmesine" neden oldu. Parse edilemeyen veya yanlış parse edilen veriyle doğru bir anomali tespiti yapmak imkansızdır.

PROCESS MINING 2.0





Tespit Stratejileri

Ayrıştırılmış (parse edilmiş) loglar üzerinde yaptığımız yüzeysel deneyler: Kurallar ve Mantık.



İki Farklı Felsefe

Kural Tabanlı (Rule-Based)

Olaylara statik gözlüklerle bakar. İnsan tarafından tanımlanmış kurallara güvenir.

- İçinde "ERROR", "FATAL" kelimesi geçen logları yakala.
- Template ID'si 15 olan log 1 dakikada 50'den fazla gelirse alarm üret.
- *Anlaması ve koda dökmesi çok kolaydır.*

Öğrenme Tabanlı (Learning-Based)

Olayların doğal akışına odaklanır. Sistemi tanımlamaz, sistemin normalini izleyip anormal olanı arar.

- Logların içindeki metinle değil, Template ID'lerinin sıklığı (frekansı) ve sırasıyla ilgilenir.
- Eğer Template 1'den sonra hep Template 2 geliyorsa, ve birden Template 8 gelirse bunu şüpheli sayar.

Kural Tabanlı Yöntemin Sınırları

Yaptığımız deneylerde kural tabanlı yapının en büyük zafiyetini net olarak gördük:

-  **Bilinmeyene Karşı Körlük:** Sadece kuralını yazdığımız hatayı bulabiliyoruz. Sistem daha önce hiç görmediğimiz yepyeni bir hata üretirse, kural setinde olmadığı için kaçırıyoruz.
-  **False Positive Çöplüğü:** İçinde "ERROR" kelimesi geçen her log sistemin çöktüğü anlamına gelmez (örn: basit bir retry işlemi). Bu durum çok fazla yanlış alarm üretiyor.
-  **Bağlam Eksikliği:** Hatalar genellikle tek bir log satırından değil, birbiri ardına gelen olaylar silsilesinden doğar. Kural tabanlı yapı bütünü göremez.



Öğrenme Tabanlı Yöntemin Avantajı



Sıklık Analizi

Logların içeriğini okumasına gerek kalmadan, sadece her bir template'in zaman penceresi içindeki görülme sıklığına (count) bakarak sistemin genel sağlığını anlayabiliriz.



Davranışsal Ayak İzi

Hiç hata barındırmayan "INFO" logları bile eğer normal sırasının dışında, çok yoğun şekilde üretiliyorsa (örn: sürekli bağlantı denemesi), öğrenme modelleri bunu anomali olarak yakalar.



Dinamik Adaptasyon

Sistem güncellendiğinde veya yeni özellikler eklendiğinde log yapısı değişir. Öğrenme tabanlı sistemler, sürekli kural güncellemek yerine yeni "normali" kendi kendine öğrenir.



Deney Sonuçları

Kriter	Kural Tabanlı Tespit	Öğrenme Tabanlı Tespit (Konsept)
Kurulum ve Uygulama	Çok hızlı ve basit (Regex / Keyword).	Ön hazırlık (Parsing) ve feature extraction ister.
Over-Clustering Etkisi	Nispeten düşük (keyword arandığı için).	Kritik! Şablonlar karışırsa model tamamen yanılır.
Yeni Hata Yakalama	Sıfır. Kuralda yoksa hata da yoktur.	Yüksek. Normalin dışındaki her sapmayı görür.
Gürültü (False Positive)	Yüksek (Her ERROR kritik değildir).	Eğitim verisinin kalitesine bağlı olarak optimize edilebilir.

Öğrenme Tabanlı: Sıklık (Frekans) Sapması



Sistemdeki tek bir Template'in (örn: bağlantı talebi) zaman pencerelerindeki görülme sıklığı. Kural tabanlı sistem bu logda "ERROR" yazmadığı için sessiz kalırken, öğrenme tabanlı sistem 10:45'teki ani *davranışsal sapmayı* anomali olarak yakalar.

Özet Çıkarımı



Hibrit Yaklaşım Şart

Sadece biri yetmez.

Ufak çaplı deneylerimiz bize gösterdi ki; **Kural Tabanlı** yaklaşımı tamamen çöpe atmak mantıklı değildir. Bilinen ve kesin olan hatalar (Örn: Yetki reddi) için hızlıca kurallar yazılmalıdır.

Ancak sistemin genel sağlığını izlemek, görünmez hataları yakalamak ve false-positive oranını düşürmek için mutlaka sağlam bir Log Parsing (Drain3) işleminden geçirilmiş veriyle beslenen **Öğrenme Tabanlı** bir anomali tespit altyapısı kurulmalıdır.



Dinlediğiniz ve kıymetli vaktinizi ayırdığınız için
teşekkür ederim.

Ahmet Ayduğan

Monitoring & DevOps